Google Summer of Code

sbi

NUMFOCUS

OPEN CODE = BETTER SCIENCE

Project Proposal for

Integrate Azula Library for Diffusion Sampling

Mentors: Jan Teusen, Nicholas Junge

Medium (~175 hours)

By

Satwik Sai Prakash Sahoo

Odisha University of Technology and Research

sahoospsatwik@gmail.com

linkedin.com/in/satwiksps

github.com/satwiksps

+91-7077910547

Table of Contents

Particulars	Page
1. About Me	02
2. My Open-source Journey with SBI	03 - 05
2.1 Why does SBI resonate with me	03
2.2 My Contributions in SBI	03 - 05
3. Abstract	05
4. Motivation	05
5. Technical Details	06 - 07
5.1. The Score-to-Mean Translation (Tweedie's Formula)	06
5.2. The Schedule Shape Bug Causing Vector Misalignment	06
5.3. Initialisation Alignment	07
5.4. Time Direction	07
6. Implementation Details	07 - 10
6.1. Class Architecture	07
6.2. SBISchedule (Schedule Adapter with shape correction)	08
6.3. SBIDenoiser (Score-to-Denoiser Translation)	08
6.4. Dependency Management	09
6.5. Note on DDPM Samplers and Schedule Variants	10
7. Weekly Timeline For Deliverables	10 - 13

8. Fallback Options and Risk Mitigations	13 - 14
9. Project Outcomes	14
9.1. Value for Users	14
9.2. Value for Maintainers	14
10. Work Commitment	14
11. Post GSoC Plans	15

1. About Me

I am an Electrical Engineering undergraduate at Odisha University of Technology and Research, Bhubaneswar, with a strong foundation in Applied Mathematics and Computer Science. My coursework includes Mathematics I, II, & III, Signals & Systems, Programming in C & Data Structures, Soft Computing, and Object-Oriented Programming.

I have a passion for writing efficient, clean code, backed by a rigorous competitive programming background. On LeetCode, I have achieved the 'Knight' badge with a peak rating of 1994, placing me in the top 5% of users globally. I also actively compete on Codeforces as a 'Pupil' (max rating 1282), where I recently secured a global rank of 2191 in Round 1072 (Div. 3).

I was selected for the Amazon ML Summer School *(August 2025 – September 2026)*, ranking among the top 3,000 candidates out of over 80,000 applicants to receive a month-long mentorship from Amazon scientists about Machine Learning.

I secured 2nd position at a national level hackathon *(October 2025)* organised by IIIT Nagpur for building a HPC enabled MLOps pipeline architecture.

I participated in the IBM QAMP *(Oct 2025 – Jan 2026),* where I collaborated with a global team to develop a Python package that integrates data engineering and ML algorithms with quantum computing which will soon be published on PyPI.

Languages: Python, C, LaTeX
Frameworks: sbi, PyTorch, NumPy, SciPy, Pandas
DevOps & Cloud: Git, Docker, Kubernetes, AWS, GitHub Actions

2. My Open-Source Journey with SBI

2.1 Why does SBI resonate with me?

My interest in SBI stems from the intersection of my academic focus on Electrical Engineering (Signal Processing) and my passion for probabilistic machine learning. Simulation-based inference has helped me for analysing complex physical systems where likelihoods were intractable.

Having worked on HPC and MLOps, I appreciate the library's modular design and its ability to democratize powerful inference methods. I want to contribute to sbi to make these advanced statistical tools more robust, efficient, and accessible to the scientific community.

2.2 My contributions in SBI

SL.	Link	Title	Description
01	#1744 Merged	Use Literal for strict typing of z-scoring options	Replaced loose string arguments with *typing.Literal* in public API to enable static type checking and catch configuration typos at development time.
02	#1711 Merged	Refactor z-scoring tests & enable MNLE support	Refactored tests to fix shape mismatches, resolved variable naming confusion, and enabled testing for Mixed Neural Likelihood Estimation (MNLE) by handling discrete data correctly.
03	#1707 Merged	Fix device mismatch in NPSE marginal computation	Resolved a crash on GPUs by ensuring the internal time tensor used for NPSE marginal statistics is created on the same device as the estimator's registered buffers.
04	#1706 Merged	Fix misleading assertion message in NPE train	Corrected an assertion error message in *NPE.train()* to accurately guide users on whether to resume training or append new simulations during multi-round inference.
05	#1705 Merged	Add max sampling time support to rejection samplers	Implemented a timeout mechanism to prevent infinite loops in rejection sampling and added the *reject_outside_prior* flag to handle highly concentrated posteriors efficiently.
06	#1703 Merged	Add 1D time-series support to Transformer Embedding	Enhanced Transformer Embedding to support 1D time-series inputs automatically and replaced complex configuration dictionaries with user-friendly keyword arguments.

07	#1695 Merged	Docs: How-to guide for time-series embedding	Authored a comprehensive guide demonstrating how to use Causal CNN and Transformer embeddings, providing a clear workflow for users with sequential data.
08	#1753 Merged	Trigger workflow when PR is marked ready for review	Optimized CI/CD pipeline to automatically trigger checks when a draft PR is marked "Ready for Review," improving developer experience.
09	#1750 Merged	Raise informative Runtime Error on empty simulations	Added guard clauses to *train()* methods across all inference families (NPE, NLE, NRE) to catch empty datasets early and provide actionable error messages.
10	#1752 Merged	Re-enable time-dependent z-scoring for Flow Matching	Re-introduced time-dependent normalization for Flow Matching estimators to ensure network inputs remain well-conditioned throughout the diffusion integration process.
11	#1749 Merged	Fix memory bloat in multi-round inference	Resolved a critical memory leak by removing unnecessary deep copies of the neural network during training loops and optimizing the storage of posterior models.
12	#1708 Merged	Default documentation should point to stable	Diagnosed incorrect redirects and guided maintainers to update *Read-The-Docs* configuration to default to stable instead of latest.
13	#1696 Closed	Support scalar time series and simplify config handling in the Transformer Embedding	Proposed enhancing Transformer Embedding to accept 1D time-series inputs directly and replacing the complex dictionary config with explicit keyword arguments for better usability.
14	#1791 Merged	Support shape broadcasting and condition sample_dim in density estimators	Refactored tensor shape handling across all density estimators to support native PyTorch broadcasting, enabling 1-to-N mappings and seamless routing of 3D condition tensors.
15	#1810 Merged	Add IID x support to PosteriorBasedPotential	Added IID support to PosteriorBasedPotential using a vectorized batched forward pass, enabling MCMC and importance sampling for NPE on IID datasets.
16	#1710 Answered	Integrating MLNE with PyMC wrapper	Identified a shape mismatch error where PyMC passed 1D arrays instead of batched tensors; provided a code fix to reshape inputs for MNLE compatibility.

17	#1725 Answered	Very concentrated posterior	Resolved rejection sampling failures due to high posterior concentration by recommending the *reject_outside_prior* flag to bypass 0% acceptance rates.

3. Abstract

The sbi (Simulation-Based Inference) library implements posterior sampling through neural-network-estimated vector fields, currently offering an Euler-Maruyama SDE solver via its Diffuser → Predictor → Corrector pipeline. This project integrates the azula library which is a collection of state-of-the-art reverse diffusion samplers (DDPM, DDIM, Euler, Heun) into sbi's VectorFieldPosterior.

The core technical challenge is bridging two fundamentally different abstractions: sbi exposes a **score function** $\nabla_{\theta_t} \log p(\theta_t|x_o)$ via VectorFieldBasedPotential.gradient(), while azula's samplers consume a **denoiser** that predicts the clean mean $\mu_\phi(x_t) \approx \mathbb{E}[X|X_t = x_t]$ and returns a DiracPosterior. I built and **verified** an adapter layer that:

- Translates scores to mean predictions using **Tweedie's formula**: $\hat{\theta}_0 = (\theta_t + \sigma_t^2 \cdot s(\theta_t, t))/\alpha_t$.
- Synchronizes noise schedules by stripping the **trailing input_shape dimensions** that sbi's mean_t_fn / std_fn append (a subtle but critical incompatibility).
- Handles initialization by forwarding sbi's mean_base / std_base as data statistics to azula's schedule-aware Sampler.init().

4. Motivation

I chose this project because it perfectly aligns with my ongoing commitment to open-source scientific software and my background in probabilistic machine learning. Having already engaged with the sbi codebase and actively contributed to building end-to-end Python toolkits in complex scientific domains, I am highly familiar with navigating rigorous API design, module lifecycles, and PyTorch internals. I am also familiar with the mathematical foundations of score-based generative models, flow matching, and denoising diffusion.

Accurate and efficient posterior estimation is a critical bottleneck for researchers relying on simulation-based inference. First-order solvers often force a compromise between precision and compute time. By integrating azula's advanced higher-order and deterministic solvers, we can dramatically shift that efficiency frontier for the entire community. I am eager to apply my background in dynamical systems and open-source engineering to deliver a production-ready,

mathematically verified integration for sbi.

5. Technical Details

5.1 The Score-to-Mean Translation (Tweedie's Formula)

sbi and azula parameterize the reverse diffusion process differently:

- **sbi** provides a **score**: $s(\theta_t, t; x_o) = \nabla_{\theta_t} \log p(\theta_t | x_o)$, via ConditionalVectorFieldEstimator.score(input, condition, t).
- **azula** expects a **denoiser** returning DiracPosterior(mean=x_0_hat), where $\hat{x}_0 \approx \mathbb{E}[X | X_t = x_t]$.

For the perturbation kernel $p(\theta_t | \theta_0) = \mathcal{N}(\alpha_t \theta_0, \sigma_t^2 I)$, Tweedie's formula gives:

$$\hat{\theta}_0 = \frac{\theta_t + \sigma_t^2 \cdot s(\theta_t, t; x_o)}{\alpha_t}$$

The sign is + (not -). I verified this empirically: the + sign produces a C2ST of 0.516 against sbi's native SDE sampler, while - produces a C2ST of 1.0 (a completely wrong distribution, with the standard deviation exploding to $\approx$ 4000). This is consistent with the mathematical derivation: the score points *uphill* in probability, which corresponds to denoising *toward* the clean data.

How azula uses the denoised mean: Inside EulerSampler.step(), azula computes:

$$z_t = \frac{x_t - \alpha_t \cdot \hat{\mu}}{\sigma_t} \qquad x_s = \frac{\alpha_s}{\alpha_t} x_t + \alpha_s \left(\frac{\sigma_s}{\alpha_s} - \frac{\sigma_t}{\alpha_t} \right) z_t$$

Substituting Tweedie's $\hat{\mu} = (x_t + \sigma_t^2 \cdot \text{score})/\alpha_t$ yields $z_t = -\sigma_t \cdot \text{score}$, which cleanly recovers the probability flow ODE update direction.

5.2 The Schedule Shape Bug Causing Vector Misalignment

For FlowMatchingEstimator (used by FMPE), the perturbation kernel is $\theta_t = (1-t)\theta_0 + t\varepsilon$, giving:

- mean_t_fn(t) = 1 - t $\rightarrow$ but the implementation appends unsqueeze(-1) **for each dimension of input_shape**
- std_fn(t) = t + \sigma_{min} $\rightarrow$ same trailing dimension expansion

This means for a scalar input t = tensor(0.5):

- mean_t_fn(t) returns shape (1,) instead of () for 1D problems.
- mean_t_fn(t) returns shape (1, 1) instead of () for 2D image-like inputs.

The same pattern appears in VPScoreEstimator and VEScoreEstimator. However, azula's samplers call self.denoiser.schedule(t) in step() and init() expecting alpha and sigma to have the same shape as t (usually a scalar ()). The mismatched shapes cause silent broadcasting errors in the Euler update formula, producing garbage samples.

The Fix: The SBISchedule adapter must strip trailing dimensions:

```
def alpha(self, t: Tensor) -> Tensor:
    val = self.estimator.mean_t_fn(t)
    while val.ndim > t.ndim:
        val = val.squeeze(-1)
    return val
```

5.3 Initialization Alignment

sbi's Diffuser.initialize() draws noise as $x_T \sim \mathcal{N}(\mu_{base}, \sigma_{base}^2 I)$ directly.

azula's Sampler.init(shape, mean, var) treats mean and var as *data* statistics and applies the schedule at t_T: $x_T \sim \mathcal{N}(\alpha_T \cdot \mu_{data}, \alpha_T^2 \cdot \mathrm{var}_{data} + \sigma_T^2)$.

By passing mean=estimator.mean_base, var=estimator.std_base**2, the azula init correctly computes the noisy initial distribution. For FlowMatchingEstimator where $\alpha_1 \to 0, \sigma_1 \to 1$, both approaches converge to $\mathcal{N}(0, I)$, which matches perfectly. I verified this by comparing the init stats between both libraries, they agree to < 0.1 in both mean and standard deviation.

5.4 Time Direction

Both libraries use the same convention: sampling goes from t_{max} (noise) to t_{min} (data).

- sbi: Diffuser.run() iterates ts from t_max to t_min.
- azula: Sampler(start=t_max, stop=t_min) generates a descending timesteps grid.
  No transformation is needed, we pass start=estimator.t_max, stop=estimator.t_min directly.

6. Implementation Details

6.1 Class Architecture

The SBIDenoiser wraps ConditionalVectorFieldEstimator directly, as this gives access to score(), mean_t_fn(), and std_fn() on a single object, avoiding coupling to the prior. The observed data x_o (condition) is passed in at construction time.

For advanced use cases (IID observations, guidance methods), an alternative from_potential constructor routes through VectorFieldBasedPotential.gradient(), which composes the estimator's

score with IID correction methods and guidance wrappers.

```
VectorFieldPosterior
├── sample(sample_with="sde")   → Diffuser → Predictor/Corrector
├── sample(sample_with="ode")   → Neural ODE (zuko)
└── sample(sample_with="azula") → AzulaDiffuser
                                  ├── SBIDenoiser(azula.denoise.Denoiser)
                                  │   ├── .schedule = SBISchedule (shape-corrected)
                                  │   ├── .estimator = ConditionalVectorFieldEstimator
                                  │   ├── .condition = x_o
                                  │   └── .forward() → Tweedie → DiracPosterior
                                  └── azula.sample.{Euler,Heun,DDIM,DDPM}Sampler
```

6.2 SBISchedule (Schedule Adapter with shape correction)

```python
class SBISchedule(azula.noise.Schedule):
    def __init__(self, vector_field_estimator):
        self.estimator = vector_field_estimator

    def __call__(self, t):
        return self.alpha(t), self.sigma(t)

    def alpha(self, t):
        val = self.estimator.mean_t_fn(t)
        while val.ndim > t.ndim:  # Strip trailing input_shape dims
            val = val.squeeze(-1)
        return val

    def sigma(self, t):
        val = self.estimator.std_fn(t)
        while val.ndim > t.ndim:
            val = val.squeeze(-1)
        return val
```

6.3 SBIDenoiser (Score-to-Denoiser Translation)

```python
class SBIDenoiser(azula.denoise.Denoiser):
    def __init__(self, estimator, condition):
        super().__init__()
        self.schedule = SBISchedule(estimator)
        self.estimator = estimator
        self.condition = condition
        self.potential_fn = None

    @classmethod
```

```
    def from_potential(cls, potential_fn):
        """Create denoiser from potential function (supports IID and guidance)."""
        denoiser = cls.__new__(cls)
        super(SBIDenoiser, denoiser).__init__()
        denoiser.schedule = SBISchedule(potential_fn.vector_field_estimator)
        denoiser.estimator = None
        denoiser.condition = None
        denoiser.potential_fn = potential_fn
        return denoiser

    def forward(self, x_t, t, **kwargs):
        if self.estimator is not None:
            score = self.estimator.score(input=x_t, condition=self.condition, t=t)
        else:
            score = self.potential_fn.gradient(x_t, time=t, track_gradients=False)

        alpha_t, sigma_t = self.schedule(t)

        # Broadcast scalar schedule to match data dims
        while alpha_t.ndim < x_t.ndim:
            alpha_t = alpha_t.unsqueeze(-1)
            sigma_t = sigma_t.unsqueeze(-1)

        # Tweedie's formula (verified: + sign is correct)
        x_0 = (x_t + sigma_t**2 * score) / alpha_t
        return DiracPosterior(mean=x_0)
```

6.4 Dependency Management

Make azula an optional dependency in pyproject.toml:

```
[project.optional-dependencies]
azula = ["azula>=0.0.1"]
```

Import in vector_field_posterior.py is guarded:

```
try:
    from sbi.samplers.score.azula import AzulaDiffuser
    AZULA_INSTALLED = True
except ImportError:
    AZULA_INSTALLED = False
```

6.5 Note on DDPM Sampler and Schedule Variants

DDPM is the only *stochastic* azula sampler, it injects noise at each step via a variance parameter $\tau = 1 - \left(\frac{\alpha_t}{\alpha_s} \cdot \frac{\sigma_s}{\sigma_t}\right)^2$. I verified that DDPM works correctly with both Rectified Flow (C2ST = 0.484) and VE schedules (C2ST = 0.392). The τ values remain positive in the correct sampling direction for all tested schedules.

VE-SDE schedules have $\alpha_t = 1.0$ for all t (variance exploding), meaning $\alpha^2 + \sigma^2$ grows to ≈ 101 at $t = 1$. Despite this, all azula samplers produce valid samples because azula's update formulas are written in terms of ratios, which remain well-conditioned.

7. Weekly Timeline For Deliverables

Community Bonding Period (May 1 - May 24)

- Deep-dive into sbi's test suite, CI pipeline, and contribution guidelines.
- Study all estimator schedule implementations: FlowMatchingEstimator (rectified flow), VPScoreEstimator, SubVPScoreEstimator, and VEScoreEstimator.
- Review azula's full sampler suite and identify which samplers are compatible with which schedule types.

Week 1 (May 25 - May 29)

Milestone: Schedule adapter with shape correction.

- Implement SBISchedule with the critical squeeze(-1) fix for mean_t_fn / std_fn trailing dimensions.
- Write unit tests for shape correctness: scalar t, 1D t, batched t, various input_shape (1D, 2D, image-like).
- Verify SNR monotonicity across all estimator types (VP, Rectified Flow).

Week 2 (June 01 - June 05)

Milestone: SBIDenoiser with verified Tweedie translation.

- Implement SBIDenoiser with the + sign Tweedie formula.
- Write numerical tests: verify that DiracPosterior output is accepted by all four azula samplers.
- Test broadcasting correctness: scalar schedule $\rightarrow$ (B, D) data, (B, C, H, W) data.
- Compare single-step output against manual Tweedie computation.
- Implement the from_potential() fallback for IID and guidance cases.

Week 3 (June 08 - June 12)

Milestone: AzulaDiffuser orchestrator and VectorFieldPosterior wiring.

- Implement AzulaDiffuser with a sampler registry and run() method.
- Add sample_with="azula" path to VectorFieldPosterior.sample() and sample_batched().
- Implement automatic routing: use estimator.score() generally, fall back to potential_fn.gradient() for IID.
- Wire up rejection sampling for reject_outside_prior=True.

Week 4 (June 15 - June 19)

Milestone: Test suite and C2ST validation on linear_gaussian.

- Create tests/azula_test.py with parametrized tests (heun, euler, ddim, ddpm).
- Test on linear_gaussian 2D: assert correct shapes and C2ST < 0.75 against ground truth.
- Test edge cases: single observation, batched observations, BoxUniform prior.
- Add azula to pyproject.toml optional dependencies.

Week 5 (June 22 - June 26)

Milestone: two_moons validation and schedule compatibility testing.

- Test on two_moons: achieve C2ST < 0.6 against native SDE sampler.
- Test azula integration with VP and VE estimators (NPSE).
- Validate DDPM's τ parameter remains positive for all schedule types in the sampling direction.

Week 6 (June 29 - July 03)

Milestone: Benchmarking framework.

- Build comprehensive benchmark comparing all azula samplers vs sbi's SDE/ODE at multiple step counts.
- Generate convergence plots: C2ST vs steps (expect Heun to converge faster than Euler due to 2nd-order accuracy).
- Generate efficiency frontier: C2ST vs wall-clock time.
- Profile adapter overhead: measure Python wrapping cost vs native sampling.

Midterm evaluation (July 6 - 10)

Deliverables:

- Complete, tested sbi/samplers/score/azula.py module.
- Modified vector_field_posterior.py with sample_with="azula" support.

- Passing test suite with >90% coverage of new code.
- Benchmark results demonstrating C2ST < 0.6 on two_moons .
- Draft PR ready for mentor code review.

Week 8 (July 13 - July 17)

Milestone: Documentation and user-facing tutorial.

- Write user-facing documentation for sbi's docs site.
- Add docstrings meeting sbi's style guide.
- Document the Tweedie formula derivation and the schedule shape issue for maintainers.

Week 9 (July 20 - July 24)

Milestone: DDPM deep-dive and stochastic sampler validation.

- Dedicated week for DDPM-specific testing across all schedule types: FM, VP, SubVP, VE.
- Verify DDPM's τ computation and noise injection for edge cases (very small step sizes, near-boundary times).
- Test DDIM with configurable η parameter ($\eta = 0 \rightarrow$ deterministic, $\eta = 1 \rightarrow$ DDPM-equivalent).

Week 10 (July 27 - July 31)

Milestone: IID observation handling and guidance composition.

- Verify azula sampling works correctly with x_is_iid=True, using the from_potential() path.
- Handle the fnpe initial distribution adjustment (init_std = \sqrt{1/N} \cdot \text{std_base}).
- Test interaction with guidance methods through the potential function routing.
- Implement save_intermediate for diffusion trajectory visualization.

Week 11 (Aug 03 - Aug 07)

Milestone: GPU testing and optimization.

- Run a full benchmark suite on CUDA.
- Optimize device transfers (ensure mean_base/std_base are on the correct device).
- Ensure torch.no_grad() is properly applied throughout.
- Test higher-dimensional problems (5D, 10D linear Gaussian).
- Run ruff linter and pyright type checker on all new code.

Week 12 (Aug 10 - Aug 14)

Milestone: Final polishing.

- Ensure all tests pass in CI.
- Write a comprehensive PR description with a benchmark results table.
- Address any remaining code review feedback.
- Update CHANGELOG.md.

Final Submission (Aug 17 - Aug 24)

- Finalize the PR and get it approved from mentors for merge.
- Write GSoC final report blog.

8. Fallback Options and Risk Mitigation

Given the mathematical complexity of bridging two independent diffusion libraries, I have identified potential risks and established concrete fallback plans to ensure the project delivers a merged, functional PR on time.

Risk 1: DDPM $\tau < 0$ for specific schedule types (e.g., SubVP)

If the variance tracking for the stochastic DDPM sampler breaks due to incompatible schedule asymptotes, I will clamp τ to the $[0, 1]$ range within the adapter. If mathematical divergence persists for a specific schedule, I will explicitly document DDPM as unsupported for that configuration. As a fallback, DDIM with $\eta = 0$ will be provided as a stable, deterministic alternative.

Risk 2: Multi-step azula samplers incompatible with sbi schedules

Advanced samplers like zABSampler or DPM-Solver variations often call schedule(time) on batched time vectors. While my SBISchedule squeeze fix handles batched inputs correctly (verified), if unforeseen internal state-tracking issues arise within azula, the PR scope will be narrowed to officially support only the thoroughly verified single-step samplers (Euler, Heun, DDIM) for the initial release.

Risk 3: VE schedule instability at extreme σ values

My prototype tests showed that Variance Exploding (VE) schedules work (achieving C2ST = 0.667), though with a wider gap than Rectified Flow. If edge-case numerical instability occurs near the data distribution ($t \to 0$), I will implement stop clamping within the sampler wrapper to avoid evaluating the neural network at near-zero σ_t regimes.

Risk 4: IID/Guidance not composable via the estimator-direct path

I have already designed and verified the from_potential() fallback constructor. The routing path using VectorFieldBasedPotential.gradient() handles all IID composition methods (like fnpe) and guidance wrappers transparently, ensuring no loss of existing sbi functionality.

Risk 5: Unforeseen Technical Delays and Timeline Overruns

If a core milestone requires more time than budgeted, I will utilize the final polishing weeks as a buffer. Also, non-blocking enhancements such as save_intermediate trajectory visualization, higher-dimensional testing, or DDIM η parameterization, can be safely decoupled and deferred to a follow-up PR. This ensures the core AzulaDiffuser integration remains stable, thoroughly reviewed, and merged well before the final GSoC deadline.

9. Project Outcomes

9.1 Value For Users

- **More accurate posteriors with fewer steps.** Higher-order solvers like Heun's method achieve the same distributional accuracy (C2ST) as Euler-Maruyama with roughly half the steps, directly reducing wall-clock sampling time.
- **Deterministic reproducibility.** DDIM and Euler produce fully deterministic samples given the same initial noise, eliminating stochastic variance between runs.
- **Drop-in API.** Users switch samplers with a single argument change: posterior.sample((1000,), sample_with="azula", predictor="heun"). No changes to training or model definition are needed.
- **Access to state-of-the-art solvers.** azula implements 10 sampler variants including DPM-Solver multi-step methods, making the cutting edge of diffusion sampling research available to the sbi community immediately.

9.2 Value For Maintainers

- **Reduced maintenance burden.** Rather than implementing each new diffusion sampler internally, sbi delegates to azula, a well-maintained library. Future azula improvements are inherited automatically.
- **Clean architectural boundary.** The adapter layer is extremely lightweight with no modifications to existing sbi classes. The estimator-direct wrapping avoids tight coupling to the potential function.
- **Gateway to azula's ecosystem.** With the adapter in place, future features like classifier-free guidance (azula.guidance) and pre-trained denoisers (azula.hub) become straightforward to integrate.

10. Work Commitment

I am fully committed to the Google Summer of Code timeline. As a university student, my summer break aligns perfectly with the GSoC coding period. I have no conflicting internships, jobs, or academic coursework during this time.

I plan to dedicate 20+ hours per week to the project. I have a reliable development setup and high-speed internet to ensure consistent communication and productivity. My top priority is to produce high-quality work, iterate on feedback, and hit all deadlines effectively.

I'm highly motivated and excited to contribute to this project, and I am confident that my availability and dedication will allow me to meet expectations effectively.

11. Post-GSoC Plans

A significant refactor or feature addition requires long-term vigilance. My journey with SBI will not end with GSoC. I intend to remain a long-term contributor to the library.

SBI is a critical tool in my own academic projects. I plan to use the improvements I build during GSoC in my future research projects, which gives me a personal stake in keeping the library robust, efficient, and scientifically accurate.

I enjoy the collaborative aspect of open source and plan to continue being an active presence in the SBI and Num Focus community.

Thank you for going through my proposal :)